THE BOOTSTRAP

A MINI GUIDE · HOW A FORTH GIVES BIRTH TO ITSELF · STONE · ITCH



FORTH PULLS ITSELF UP BY ITS OWN BOOTSTRAPS

The chicken, the egg, and the seed

- **The problem**
  Forth's reader is written in Forth. So who reads it first?
- **Forth-in-Stone: a birth you can watch**
  STONE, CLAY, MIRROR: body, hands, mind. Every cell drawn.
- **ITCH: the same birth, full size**
  32 code words, a 100-byte seed, a genome, a bone set by hand, one word that hands over.
- **The egg**
  A grown ITCH packs itself into a ROM and is born again with no seed at all.



JIMMY SÖDERMAN · UPPSALA, SWEDEN · OCTOBER 2026

BEFORE WE START

Who reads the reader?

A Forth is written in Forth. That sounds like a riddle. Here is how the riddle is solved, step by step, in two systems you can run.

JIMMY SÖDERMAN · UPPSALA, SWEDEN · OCTOBER 2026

Most of a Forth is Forth: the compiler, the control flow, even the outer interpreter, the loop that reads your text. You can read the whole system in its own language and change any part of it.

The catch: to read Forth you need a Forth reader, and the reader we want is written in Forth. Who reads it the first time? Every Forth solves this once, at birth. The word for it is **bootstrap**: pulling yourself up by your own bootstraps. Computers still say "boot" because of it.

The trick

Write a tiny reader in the host language (JavaScript here, machine code on real hardware). It reads the first pages of Forth. Those pages define the real reader, in Forth. Then the tiny reader hands over and is never used again.

A small reader, used once, to read the big reader.

One rule: nothing from outside

Each stage may use only what the stage before it made. Nothing is slipped in later. That is the difference between a **birth** and an **assembly**, and you can check it at every step in this guide.

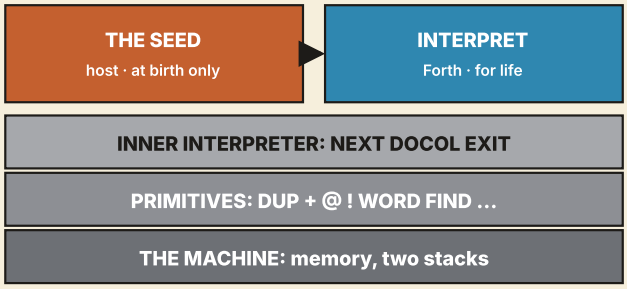


FIG. 1 *The four layers of every Forth here. The bottom three stay in the host. The top one starts as a seed and ends as Forth.*

LAYER	JOB	WRITTEN IN
Machine	memory, data stack, return stack	host
Primitives	the fixed reactions: math, memory, stacks, reading	host
Inner interpreter	runs a thread, cell by cell	host
Outer interpreter	reads text, runs or compiles each token	**host, then Forth**

TWO FORTHS, ONE STORY

Forth-in-Stone: a teaching Forth, truly indirect-threaded, its memory made of JavaScript objects. Born in three visible steps: STONE, CLAY, MIRROR.

ITCH: the full 1985-style machine, bytes in 64K. Born the same way, only smaller and faster: code words, a seed, BOOT, a handover.

THE WHOLE STORY ON ONE PAGE

The birth at a glance

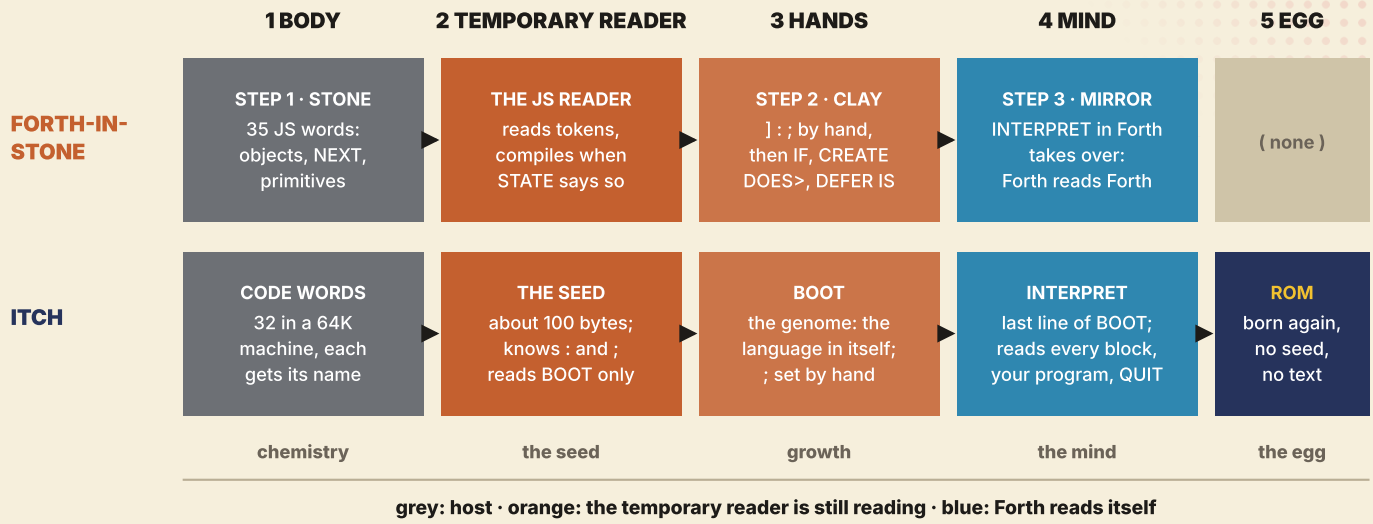


FIG. 2 *The same five stages in both Forths. The bottom row of words is the Biology name ITCH uses for each one.*

Read the picture left to right

- **Body.** Machinery only. It can run words. It cannot make them.
- **Temporary reader.** Host code that reads text, just enough to get started.
- **Hands.** Forth builds its own compiler and its strong words, using only what exists.
- **Mind.** Forth's own reader takes over. From now on, Forth reads Forth.
- **Egg.** (ITCH only.) The grown system is saved and can be born again without the seed.

What is the same, what differs

The **order** is the same in both, and so is the **handover**: one word, `INTERPRET`, read by the old reader, starts the new one.

What differs is **how long the temporary reader lives**. In Stone you use it for two whole steps, so you can watch the hands being made. In ITCH the seed lives only while BOOT is read.

WHERE THE BONES ARE

Some words cannot be made with words that do not exist yet. They are set by hand, cell by cell. Stone sets three: `]` `:` `;`. ITCH sets one: `;`. Everything else grows.

How to read this guide

Pages 4–7	Forth-in-Stone, every step drawn
Pages 8–9	ITCH: the same birth in bytes
Page 10	the egg, things to try, words to know

Stone: the body

STEP 1 · STONE
IT CAN RUN WORDS, BUT NOT MAKE THEM

A word is an object

Every word has a **name** and a **code**. A primitive's code is a JavaScript function. A colon word's code is `docol`, and it also has a **body**: a list of words.

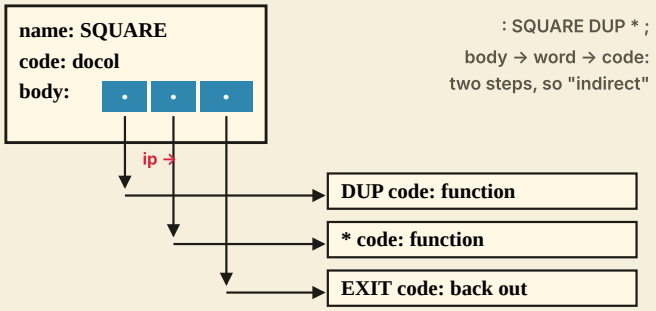


FIG. 3 *A thread holds words; each word holds its code. That is indirect threading, made of objects instead of bytes.*

The inner interpreter: 9 lines

INNER INTERPRETER · JS

```
const EXIT = { name: 'EXIT',
  code() { [th, ip] = R.pop(); } };
const LIT  = { name: 'LIT',
  code() { S.push(th[ip++]); } };
const docol = w => {
  R.push([th, ip]); th = w.body; ip = 0; };
function execute(w) {          // run one word
  const base = R.length; w.code(w);
  while (R.length > base) {    // NEXT
    const x = th[ip++]; x.code(x); } }
```

PART	DOES
`docol`	save where we are on R, enter the body
`NEXT`	take the word at ip, step ip, run its code
`EXIT`	pop R: back to where we were
`LIT`	the next cell is a number: push it

The outer interpreter: a decision tree

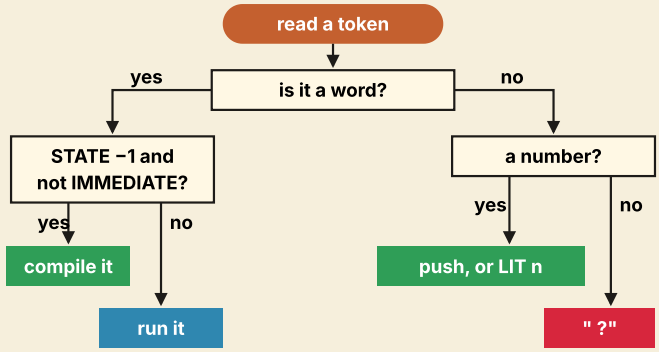


FIG. 4 *Every Forth reader in this guide makes these same choices. Remember the shape: you will see it again in Forth.*

WHAT STEP 1 CAN DO

Calculate: `2 3 + .` → 5. Use the stack: `5 DUP * .` → 25. Make a variable by hand: `WORD X (CREATE) 0 ,`. Tick a word: `' DUP .` → DUP.

What it cannot do: make a colon word. There is no `:` and no `;`. A body without hands.

Clay: the bones by hand

STEP 2 · CLAY
FORTH BUILDS ITS OWN COMPILER

There is no `:` yet, so the first three words are laid down cell by cell, with seven builder primitives that already exist:

WORD	DOES
`WORD`	reads the next name from the text
`'`	finds an existing word (tick)
`HEADER`	starts a new word, hidden
`COMPILE,`	adds a word to the new body
`LITERAL`	adds `LIT n`: a number
`REVEAL`	makes the new word findable
`IMMEDIATE`	marks it: runs even while compiling

THE THREE BONES · FORTH

```
WORD ] HEADER  -1 LITERAL ' STATE COMPILE,
  ' ! COMPILE,  ' EXIT COMPILE,  REVEAL
WORD : HEADER  ' WORD COMPILE,
  ' HEADER COMPILE,  ' ] COMPILE,
  ' EXIT COMPILE,  REVEAL
WORD ; HEADER  ' EXIT LITERAL
  ' COMPILE, COMPILE,  ' REVEAL COMPILE,
  0 LITERAL ' STATE COMPILE, ' ! COMPILE,
  ' EXIT COMPILE,  REVEAL IMMEDIATE
```

The same three words, as cells

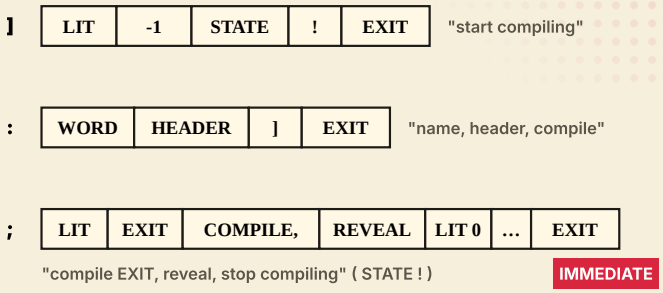


FIG. 5 *Each bone is a short body of cells. Read `;` carefully: `LIT EXIT COMPILE,` means "put EXIT into the word being built".*

Why `;` must be IMMEDIATE

While a word is being built, the reader compiles everything it meets. `;` is the one word that must *run* instead, to finish the word. IMMEDIATE is that exception.

Watch it work: `: SQUARE DUP * ;`

#	TOKEN	WHAT HAPPENS
1	`:`	runs: WORD reads SQUARE, HEADER starts it (hidden), `]` sets STATE −1
2	`DUP`	STATE is −1: the reader compiles it into the body
3	`*`	compiled the same way
4	`;`	IMMEDIATE, so it runs: adds EXIT, REVEAL, STATE 0
5	`5 SQUARE .`	SQUARE is found now: prints 25

WHO DOES THE COMPILING?

The *words* of the compiler are Forth now. But the *loop* that meets `DUP` and compiles it is still the JavaScript reader, steered by STATE. Step 3 replaces that loop.

Creeping out of the shell

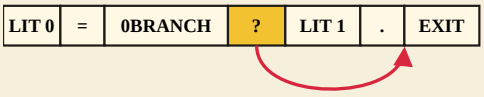
With `:` in its hands, Forth grows everything else from Forth. Three kinds of growth matter most for the birth: **control flow**, **words that make words**, and **words you can rewire**. The last two, `CREATE DOES>` and `DEFER IS`, are the open doors: through them Forth can become anything, including its own reader.

CONTROL FLOW
A HOLE NOW, FILLED LATER

IF THEN · FORTH

```
: IF   ['] 0BRANCH COMPILE,
       HERE 0 COMPILE, ; IMMEDIATE
: THEN HERE SWAP PATCH ; IMMEDIATE
```

: ZERO? 0 = IF 1 . THEN ;



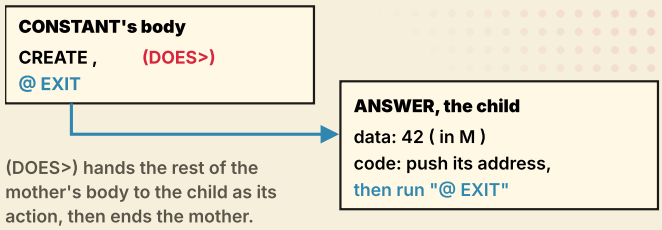
IF leaves a hole (yellow) and its place on the stack. THEN fills it with HERE: "jump to just after me". PATCH writes into a cell already built.

FIG. 6 *Control flow is just words that write cells and fix them later. ELSE, BEGIN, UNTIL, WHILE, REPEAT work the same way.*

CREATE DOES>
WORDS THAT MAKE WORDS

FORTH

```
: CREATE   WORD (CREATE) ;
: DOES>    ['] (DOES>) COMPILE, ; IMMEDIATE
: CONSTANT CREATE , DOES> @ ;
42 CONSTANT ANSWER   ANSWER .   \ 42
```



(DOES>) hands the rest of the mother's body to the child as its action, then ends the mother.

FIG. 7 *A defining word in two halves: CREATE makes the child; the part after DOES> becomes what the child does.*

DEFER IS
A PLUG YOU CAN REWIRE

FORTH

```
: DEFER  CREATE 0 , DOES> @ EXECUTE ;
: IS     ' >BODY ! ;
DEFER GREET   ' HI IS GREET   GREET  \ runs HI
```

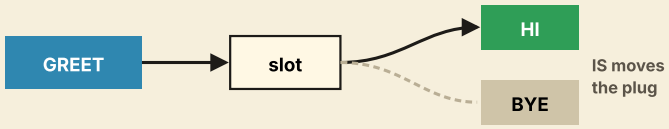


FIG. 8 *A DEFER is a CONSTANT that holds a word and runs it. IS changes which word, at any time.*

WHY THESE FOUR WORDS MATTER FOR THE BIRTH

In step 3, the new reader asks a DEFER, `UNKNOWN`, what to do with a token it does not know. So the strong words are not extras: they are how Forth will hand *you* the keys to its own reader.

Mirror: Forth reads Forth

STEP 3 · MIRROR
METACIRCULAR: FORTH'S OWN READER

INTERPRET · FORTH

```
DEFER UNKNOWN  ' NOT-FOUND IS UNKNOWN
: INTERPRET ( -- )
  BEGIN WORD ?DUP WHILE              ( token )
    FIND ?DUP IF                     ( word flag )
      1 = STATE @ 0 = OR
      IF EXECUTE ELSE COMPILE, THEN
    ELSE NUMBER? IF STATE @ IF LITERAL THEN
         ELSE UNKNOWN THEN
    THEN
  REPEAT ;
INTERPRET
```

LINE	THE SAME CHOICE AS FIG. 4
`WORD ?DUP WHILE`	read a token; stop at the end
`FIND ?DUP IF`	is it a word? flag 1 = immediate
`1 = STATE @ 0 = OR`	immediate, or not compiling → run
`NUMBER? IF … LITERAL`	a number: push, or compile LIT n
`UNKNOWN`	neither: ask the plug

The handover, on the call stack

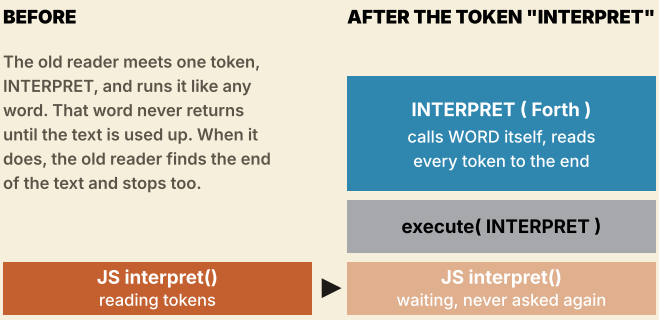


FIG. 9 *No switch, no flag. The old reader simply calls the new one, and the new one keeps the text.*

```
DEFER UNKNOWN …
: INTERPRET … ;
INTERPRET
```

```
: SQUARE DUP * ;
5 SQUARE .
… to the end
```

orange: read by JavaScript · blue: read by Forth

FIG. 10 *The same text file, two readers. The line between them is one token.*

The proof: change the reader from Forth

PROOF · FORTH

```
: SHRUG ( token -- ) DROP 0 . ;
' SHRUG IS UNKNOWN
HELLO WORLD           \ 0 0 , no error
```

The JavaScript reader would stop at `HELLO ?`. Forth's reader prints 0 because Forth said so. The reader is now yours to change, in Forth.

STEP	HOST	FORTH
1 STONE	engine, primitives, reader loop	—
2 CLAY	engine, primitives, reader loop	compiler, control flow, strong words
3 MIRROR	engine, primitives	all of that **and** the reader loop

The helpers `WORD` `FIND` `NUMBER?` stay primitives in every step, as in ITCH and in most real Forths: the loops are Forth, the helpers are machine routines.

ITCH: names, then the seed

STAGE 1 · THE BODY

32 CODE WORDS IN A 64K MACHINE

Power on: the kernel clears 64K of memory and gives each code word a header, reading the names with its own `WORD`. Biology: the first reactions get their names.

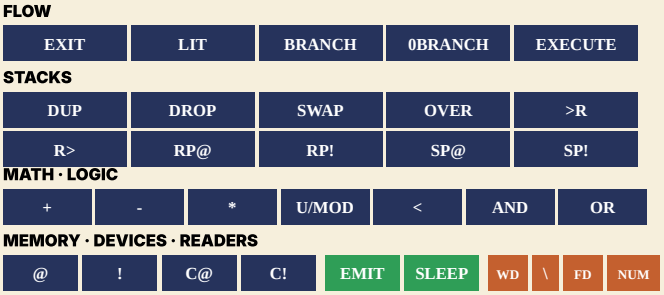


FIG. 11 *26 CPU words (navy), 2 devices (green), 4 readers (orange: WORD \ FIND NUMBER), plus one runtime, DOCOL. That is the whole body.*

The four readers stay code words for one reason: **the seed needs them** before any Forth exists.

STAGE 2 · THE SEED

ABOUT 100 BYTES OF MACHINE CODE

THE SEED · JS

```
function interpret() {
  let comp = 0;                 // a private flag
  for (;;) {
    const a = word(32); if (!m[a]) return;
    if (is ":") { header(word(32));
      comma(DOCOL); comp = 1; continue; }
    if (is ";") { comma(xtEXIT);
      comp = 0; continue; }
    const h = find(a), side = m[h+2] & 0xC0;
    if (h && (!comp || a builder)) run(cfa(h));
    else if (h) comma(cfa(h));
    else if (comp) {
      comma(xtLIT); comma(number(a)); }
    else push(number(a));
  } }
```

THE SEED MEETS	IT DOES
`:`	header for the next name, DOCOL, flag on
`;`	compile EXIT, flag off
a word, flag off	run it
a word, flag on	compile it, unless it is a BUILDER or BOTH: run it
a number	push it, or compile `LIT n`

THE SEED'S ONLY EXCEPTION

The seed has no IMMEDIATE. It looks at the header's two side bits: a **BUILDER** word (like `IF`) or a **BOTH** word (like `(`) runs even while compiling. That is how BOOT can use `IF THEN` inside the very words that build the language.

The genome, the bone, the handover

STAGE 3 · BOOT
THE LANGUAGE WRITTEN IN ITSELF

The seed reads BOOT top to bottom. Each section may use only the sections above it:

- system cells: DP LATEST >IN BASE ...
- growth: HERE ALLOT , LIT, ' BUILDER BOTH
- stack helpers: NIP ROT R@ NEGATE ...
- control flow: IF THEN ELSE BEGIN UNTIL ...
- flags, memory, output, strings
- CREATE DOES>: words that make words
- DEFER IS: words you can rewire
- the reader: receptors, RECOGNIZE
- INTERPRET and]: two loops, no STATE
- gene groups, CSP, then : in Forth
- ; laid down by hand: the bone
- INTERPRET: hand over

FIG. 12 *BOOT is the genome: read once, top to bottom, by the seed. Orange is read by the seed; the last blue dot is where Forth takes over.*

A first trick: `LIT LIT , ,`

BOOT · FORTH

```
: ,     HERE ! 2 ALLOT ;
: LIT,  LIT LIT , , ;   \ compile a literal
```

Inside `LIT,` the first `LIT` pushes the next cell as data, and that cell is `LIT` itself. So `LIT,` lays down `LIT`, then the number. The language uses its own machinery to describe itself.

The bone: `;` set by hand

Near the end, Forth defines its own colon: `: : !CSP NAMED DOCOL , ] ;`. But in ITCH a word can be found while it is being defined, so a `;` that ended in `;` would find itself, half built. It is laid down cell by cell:

THE BONE · FORTH

```
NAMED ;  DOCOL ,  ' ?CSP ,
  ' LIT , ' EXIT , ' , ,      \ compile EXIT
  ' R> , ' DROP ,             \ leave ]
  ' EXIT ,  BUILDER
```

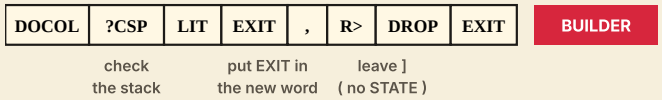


FIG. 13 *The one bone ITCH cannot grow. Built by hand, it reads the same to the seed and to Forth, so Forth can read all of BOOT again.*

STAGE 4 · HANDOVER
ONE WORD, THE LAST LINE

THE LAST LINE OF BOOT

```
INTERPRET  \ hand over: the bootstrap
           \ reader stops here; Forth reads
           \ every block after this
```

Two loops, one cell apart

BOOT · FORTH

```
: INTERPRET  BEGIN BL WORD DUP C@ WHILE
    RECOGNIZE @     EXECUTE REPEAT DROP ;
: ]          BEGIN BL WORD DUP C@ WHILE
    RECOGNIZE 2 + @ EXECUTE REPEAT DROP ;
```

Every token's type holds two actions: cell 0 for acting, cell 2 for building. `INTERPRET` takes cell 0, `]` takes cell 2. Which loop is running *is* the state, so ITCH needs no STATE variable and no IMMEDIATE test. `[` and `;` leave `]` with `R> DROP`.

Grown up, and born again

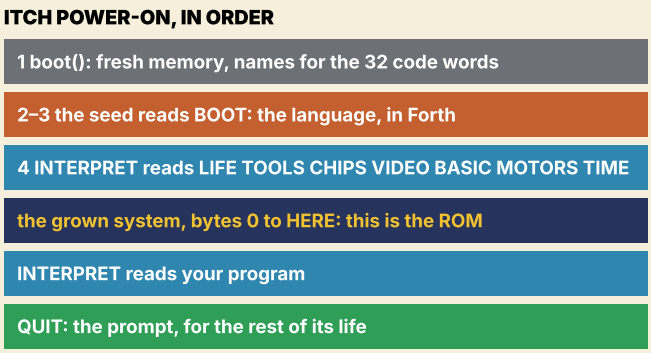


FIG. 14 *Only the orange stage is read by the seed. Everything blue is read by Forth.*

The egg: no seed needed twice

Save ROM writes the grown system (bytes 0 to `HERE`) to a file. **Boot ROM** powers on from it with no seed and no source: the code words, plus memory that Forth built. The reset routine finds `QUIT` and runs it.

Biology: a grown body packs itself into an egg that grows the same body again. On real hardware this is how ITCH would ship: a ROM chip, no text, no seed.

TESTS ITCH PASSES

- The ROM runs a demo exactly like the source does.
- Forth re-reads its own blocks from a ROM holding only the language, with the same result.
- That ROM boots again.

Same birth, two sizes

STAGE	STONE	ITCH
Body	35 JS words, objects	32 code words, bytes
Temporary reader	JS reader, steps 1–2	the seed, BOOT only
Bones by hand	`] : ;`	`;`
Run or compile?	STATE, IMMEDIATE	two loops, sides
Handover	`INTERPRET`, end of MIRROR	`INTERPRET`, end of BOOT
Egg	—	Save ROM, Boot ROM

Try it yourself

- ☐ Stone: press steps 1, 2, 3 and compare what each can do.
- ☐ Stone step 3: plug `SHRUG` into `UNKNOWN`, type a word that does not exist.
- ☐ ITCH: find the hand-made `;` and the last line of BOOT.
- ☐ ITCH: open EXPLORE and STEP a line. Watch Forth's `INTERPRET` read it, one NEXT at a time.
- ☐ ITCH: Save ROM, then Boot ROM. Type at a Forth born without a seed.

Words to know

Bootstrap	starting a system with a small helper that is then thrown away
Primitive	a word written in the host, not in Forth
Inner interpreter	runs a thread: NEXT, DOCOL, EXIT
Outer interpreter	reads text; runs or compiles each token
Seed	the temporary outer interpreter
Bone	a word set by hand because it cannot be grown
Handover	the moment Forth's own reader takes over
Metacircular	a language whose reader is written in itself